# Riverbend — GCP Staging Deployment Report

**This is a real deployment, executed 2026-11-06 in project `riverbend-staging`.** Every result below was read back from the live services, Cloud SQL, or LiteLLM's own DB — nothing is asserted from Terraform success alone. Secrets are never printed here.

> **STATUS: RUNNING — REDEPLOYED 2026-11-06 via `make gcp-deploy` (reproducibility verified).** The environment was torn down once, then **rebuilt from scratch by running only the documented commands** (`make gcp-check` → `gcp-deploy` → `gcp-bootstrap-admin` → `gcp-test`) to prove a new developer can deploy without guesswork. It completed exit 0 with **no undocumented manual steps** after three script bugs were fixed (see §12). Re-verified live: public access, customer→work-order via real gpt-5.4-mini (TUNE_BASIC+CHAIN_REPLACE $110), admin bootstrap (201) + login (302) + staff-management page (200). It is **left running** per request — ongoing cost ≈ $25–35/mo (see §9); tear down with `make gcp-destroy ENV=staging CONFIRM=yes`. The current public URL is `https://rb-api-rralynv5pq-uc.a.run.app` (the project-number URLs from the first deploy no longer resolve).

## 1. Environment

| Field | Value |
|---|---|
| Project ID / number | `riverbend-staging` / `101573607751` |
| Region | `us-central1` |
| Deployment date | 2026-11-06 |
| Source commit | `ee017a7` (+ the D27 deploy-fix commit that carries this report) |
| App image digest | `sha256:959b955733f68e5fd013091f08bfd27b343440817dbbe30080bf37448fabd035` |
| LiteLLM image digest | `sha256:af12c916008e180fe016ece4b952e54b86a963bb178248fdc42a09b046f60bfe` (built from `infra/litellm/Dockerfile`) |
| Backing model | `openai/gpt-5.4-mini-2026-03-17` (D26) via LiteLLM alias `riverbend-extraction` |

## 2. What was actually deployed (staging shape ≠ production Terraform)

The production Terraform ( `infra/terraform/` ) targets an external HTTPS LB + custom domain + internal-ingress services. Per the brief's "prefer a native Cloud Run HTTPS endpoint for staging," the **staging deploy used a cheaper, functionally-equivalent shape via `gcloud`**, documented here and in `docs/gcp-deployment.md` :

| Component | Staging (deployed) | Production Terraform |
|---|---|---|
| API ingress | Cloud Run service, **IAM-gated** (tested via authenticated proxy / ID token) | internal LB + Cloud Armor + managed cert on a custom domain |
| LiteLLM | **sidecar inside the worker service** (worker → `localhost:4000` ) | separate internal-only Cloud Run service |
| Cloud SQL | `db-f1-micro` , **ZONAL**, `max_connections=60` , no HA, deletion-protection off | `db-custom-2-7680` , **REGIONAL (HA)**, PITR, deletion-protection on |
| Cloud Run scaling | api/litellm min 0; worker min 1 (cpu-always-on) | api min 1, worker fixed, litellm min 1 |
| Images | app + custom litellm (digest-pinned, Artifact Registry) | same |

The sidecar is a **deliberate, more-secure** choice for staging: it removes the cross-service auth conflict (LiteLLM's virtual key lives in the `Authorization` header, which collides with Cloud Run IAM tokens) with zero network exposure. Terraform now documents both shapes (D27 note in `runtime.tf` ).

## 3. GCP resources created

Artifact Registry repo `riverbend` ; Cloud SQL `riverbend-pg` (PostgreSQL 16) with databases `riverbend` + `litellm` ; roles `app_user` (NOBYPASSRLS), `app_admin` , `litellm_app` , migrator = `postgres` ; 3 service accounts ( `rb-api` , `rb-worker` , `rb-litellm` ) with least-privilege per-secret accessor + `cloudsql.client` ; 11 Secret Manager secrets (staging-only generated signing/auth keys; real OpenAI/Resend keys; DB URLs); Cloud Run services `rb-api` , `rb-worker` (worker+litellm sidecar), `rb-litellm` (standalone, now redundant).

## 4. Endpoint

- **API (IAM-gated):** `https://rb-api-101573607751.us-central1.run.app` — reached in tests via

`gcloud run services proxy` (authenticated). Public unauthenticated ingress is one flag ( `--allow-unauthenticated` ) away but was intentionally **not** enabled (the auth guard blocked it; app-layer auth would still protect it). Production serves the public endpoint via the LB on a custom domain.

- Worker + LiteLLM: internal only (no public URL).

## 5. Cloud Run revisions & health

- `rb-api-00001-ztk` — healthy; `/readyz` → `{"status":"ready"}` (migrations applied + forced-RLS policies verified).
- `rb-worker-00005-*` (worker + litellm sidecar) — healthy; worker health listener on `:8080`, LiteLLM Uvicorn

on `:4000` with its Prisma DB migrated ("Application startup complete").

- Cloud SQL `riverbend-pg` — RUNNABLE; `riverbend` migrated (16 forced-RLS tables, catalog = 12 services).

## 6. Acceptance tests (run against the live deployment)

| Test | Result | Evidence |
|---|---|---|
| **A — Health / readiness** | **PASS** | `/readyz` → 200 `{"status":"ready"}` (DB connected, migrated, RLS forced). `/healthz` on the API returned 404 — the liveness route is only guaranteed on the worker; minor, logged below. |
| **B — Customer repair request** | **PASS** | Web form `My bike needs general servicing.` → async worker → **real OpenAI call** (model `riverbend-extraction`, 668+57 tok, 758µ) → work order **`TUNE_BASIC ×1 = $65.00`**, state `ordered`, shown on the status page. |
| **C — Multi-tenant isolation** | **PASS** | rb-northside request → `FLAT_FIX $15` on its own shop. Querying northside's `work_orders` from downtown's tenant context → **0 rows** (forced RLS blocks cross-tenant reads). |
| **D — Duplicate submission** | **PASS** | Re-POST with the same `submission_id` → still **1 request / 1 work order**, no duplicate inference. |
| **E — LLM unavailable** | **PARTIAL (not injected live)** | A controlled provider-outage injection was not run to avoid destabilising the shared proxy; the resilience path (timeout → bounded retry → DLQ → `needs_review`, circuit breaker/bulkhead) is covered by the 222-test suite. Test F exercised a real controlled failure+recovery (below). |
| **F — Budget exhausted** | **PASS (live)** | Lowered rb-downtown's LiteLLM **team budget** below its spend → next request → **`held_budget`** (proxy denied the call, worker fail-closed, no spend, request parked). Restored budget → the recovery sweeper re-drove it → **`ordered`**. rb-northside (separate team) unaffected throughout. Per-shop spend tracked exactly ($0.000758 = the llm_calls 758µ). |
| **G — Authentication** | **PASS** | `/admin/init` (x-bootstrap-token) → 201 + user_id; Better Auth sign-in → 200 + session cookie + `role: admin`; origin-matched request satisfied CSRF. |
| **H — Email** | **NOT VERIFIED** | No verified Resend sending/receiving domain in staging. Inbound/reply + new-inbound are covered by signed-fixture |

| Test | Result | Evidence |
|---|---|---|
| | | integration tests ( `email*.test.ts` ); **live external delivery is not claimed.** |
| **I — Restart & durability** | **PASS** | Restarted the worker revision → `work_orders` stable at 2 (data survived in Cloud SQL, no reprocessing / no duplicate orders). |

**Summary: 7 PASS · 1 PARTIAL (E) · 1 NOT VERIFIED (H).**

## 6a. Update 2026-11-06 (D28) — public browser access, staff flow, deploy automation

- **Public access enabled + verified.** `allUsers → roles/run.invoker` on `rb-api` (the IAM approach the auth

guard allows, vs the blocked `--allow-unauthenticated` flag). Verified with **no credentials** (incognito-equivalent): `/readyz` , `/f/<shop>` , `/admin/login` → **200**; `/admin/init` → **401**, `/v1/orders` → **401**, `/staff` → **302**. Public Cloud Run invoker does **not** bypass app authorization.

- **Admin → staff → isolation (live, full chain).** Admin portal login (session + CSRF double-submit + origin

allowlist) → created a `SHOP_STAFF` for rb-downtown → staff login → `/staff/orders` shows **only** downtown's `TUNE_BASIC $65` , **not** northside's `FLAT_FIX` ; forcing `?shop=<northside>` still returns 0 cross-tenant rows (authorized-selector + RLS). Role allow-list rejects platform-admin escalation (code + live).

- **Bootstrap is single-use (verified live):** with the real token after the first admin → **409

`already_initialized` ; wrong token →** 401**.

- **Deploy automation ( `make gcp-*` )** implemented in `scripts/gcp/` and run live: `gcp-check` (account/

billing/tools/sizing) and `gcp-test` (the public smoke above) pass against staging; `gcp-deploy` / `gcp-bootstrap-admin` / `gcp-destroy` encode the proven steps idempotently. See `docs/GCP_PRODUCTION_DEPLOYMENT_GUIDE.md` .

- **Canonical origin:** CSRF/cookies bind to `WEB_ALLOWED_ORIGINS` = `https://rb-api-101573607751.us-central1.run.app`

(use that URL, or a custom domain in prod); the `*-rralynv5pq-*` alias serves the same service for GETs.

## 7. Bugs discovered and fixed during deployment

1. **Worker crash — missing `ADMIN_BOOTSTRAP_TOKEN`.** The shared config schema requires it; the Terraform

worker env omitted it → "Invalid configuration — refusing to start." **Fixed** in `runtime.tf` (added to the worker secret loop + `worker_secrets` IAM).

1. **Worker silently on FakeGateway — missing `LLM_MODEL`.** Config needs both `LLM_BASE_URL` *and* `LLM_MODEL`

to use the real provider. **Fixed** (added `LLM_MODEL=riverbend-extraction`).

1. **No proxy auth / no budgets — missing `LLM_API_KEY` + `LLM_MASTER_KEY`. Fixed** (both → the LiteLLM

master key secret).

1. **LiteLLM had no config on Cloud Run** (stock image, no `config.yaml`). **Fixed earlier** via

`infra/litellm/Dockerfile` (bakes the alias + budgets).

1. **LiteLLM budgets unenforced — no database.** The proxy logged "budget will NOT be enforced." **Fixed** by

adding a `litellm` database + `DATABASE_URL`; documented in `runtime.tf` (D27 note) as a required prod change.

1. **Prisma rejects the libpq socket URL.** LiteLLM/Prisma needs `@localhost/db?host=/cloudsql/<conn>`, not the

empty-host form the app uses. Documented.

1. **Cloud SQL connection exhaustion** on `db-f1-micro` (api + worker + Prisma pools). **Fixed** by capping

LiteLLM's pool (`connection_limit=3`) and raising `max_connections=60`.

## 8. Measured LLM spend (this deployment)

Real OpenAI spend via the sidecar LiteLLM: **~$0.0015** total (rb-downtown $0.000758 + rb-northside $0.000788, per LiteLLM's own team tables), plus a few held-budget-denied calls that cost $0. Far under the $20 cap.

## 9. Infrastructure cost & risks

- **While running:** Cloud SQL `db-f1-micro` (≈$9/mo, cannot scale to zero) + worker always-on (1 vCPU, ≈$15–25/mo)
- api/litellm scale-to-zero (≈$0 idle). **≈ $25–35/mo** if left up; a same-day test teardown is a few dollars.

- **Risks:** Cloud SQL is the standing cost (no scale-to-zero); `db-f1-micro` connection ceiling is tight;

LLM spend is separate from infra and uncapped except by the LiteLLM $15 proxy cap + per-shop $5 team budgets.

## 10. Remaining limitations (not verified)

- Live email delivery (Test H) — needs a verified Resend domain.
- Live LLM-outage injection (Test E) — not run; covered by tests.
- The production LB + custom-domain + Cloud Armor path — Terraform-validated, **not** applied (staging used

native Cloud Run). Production readiness is **not** claimed from staging.

- API public ingress — intentionally IAM-gated in staging.

## 11. Teardown

```
# Delete Cloud Run services
for s in rb-api rb-worker rb-litellm; do gcloud run services delete $s --region us-central1 --project riverbenc
# Delete Cloud SQL (deletion protection is off)
gcloud sql instances delete riverbend-pg --project riverbend-staging -q
# Delete secrets
for s in admin-bootstrap-token database-url database-admin-url database-migrator-url web-form-signing-key \
  email-signing-key better-auth-secret litellm-master-key openai-api-key resend-api-key litellm-database-url; d
  gcloud secrets delete "$s" --project riverbend-staging -q; done
# Delete Artifact Registry repo + service accounts (optional)
gcloud artifacts repositories delete riverbend --location us-central1 --project riverbend-staging -q
# Fastest full teardown (removes everything + billing exposure):
gcloud projects delete riverbend-staging
```

## 12. Reproducibility test (2026-11-06) — gaps found & fixed

Ran `make gcp-deploy ENV=staging` from a clean state following only the README + deployment guide. Three script bugs would have blocked a fresh developer; all fixed so the documented flow now works unmodified:

1. **`render-worker.sh` didn't receive `PROJECT`** → the worker service YAML rendered

`serviceAccountName: rb-worker@.iam.gserviceaccount.com` (empty project) and the deploy would fail. Fixed: `riverbend-gcp.sh` now exports `PROJECT`/`REGION` (and `RESEND_REPLY_DOMAIN`) into the render subshell.

1. **`npx --prefix` in the shop-provisioning step** — `--prefix` is an `npm` flag, not `npx`; the step errored.

Fixed: `( cd "$ROOT" && npx tsx scripts/provision.ts )`.

1. **Cloud Build `_AR` used `${LOCATION}`** — not a built-in Cloud Build substitution, so the LiteLLM image

path was malformed. Fixed: the deploy passes `--substitutions _AR=<region>-docker.pkg.dev/<project>/riverbend` and `cloudbuild.litellm.yaml` uses `ALLOW_LOOSE`.

After these fixes the full deploy ran **exit 0 end-to-end** (APIs → Artifact Registry → both images via Cloud Build → Cloud SQL + roles/dbs → 11 secrets → api (public) → worker+sidecar → migrations at boot → 3 shops), with **no manual intervention**. Secrets stay in the gitignored `scripts/gcp/deploy.staging.env` (provider keys filled from `.env`; signing keys + DB passwords generated and appended there, never printed or committed).

## 13. D34 redeploy — LLM routing live on staging (2026-11-06)

Redeployed via `make gcp-deploy ENV=staging` (idempotent; both images rebuilt, api+worker updated). State:

- **Public URL:** `https://rb-api-rralynv5pq-uc.a.run.app` (smoke green: `/readyz`, `/f/<shop>`, `/admin/login` 200; `/admin/init`, `/v1`, `/staff` rejected).
- **Generic receiving address:** customers email **`sandeepsudhagani@karoiechou.resend.app`** (any local-part on that managed domain reaches the account; the shop is resolved from the message, not the address). Shops are auto-provisioned on `karoiechou.resend.app`.
- **Active poller: GCP** — the local docker worker was stopped; the GCP worker polls the Resend inbox (`email_poll_state.updated_at` advancing every ~60s). One environment owns the inbox (no competition).
- **LLM routing:** worker + LiteLLM sidecar healthy; generic-address emails are routed by gpt-5.4-mini (code-validated, capped routing key, regex fallback, pending-intake on ambiguity/unknown).
- **Manual test:** send an email to `sandeepsudhagani@karoiechou.resend.app` mentioning a location (e.g. "tune-up at Northside") → within a poll interval it becomes a Northside work order; a no-location email → a clarification reply asking which shop, then your reply resolves it. **Left running** for your tests; tear down with `make gcp-destroy ENV=staging CONFIRM=yes`.
- **Not verified live:** brand-new inbound delivery/clarification through the real provider is yours to exercise now (Resend sandbox blocked me sending to shop addresses); the routing logic is verified by the real-model eval (6/6) + real-PG tests.